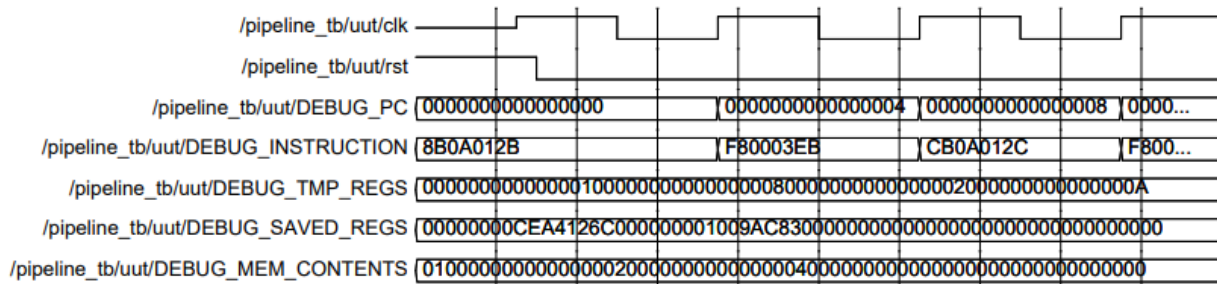


Lab4 report

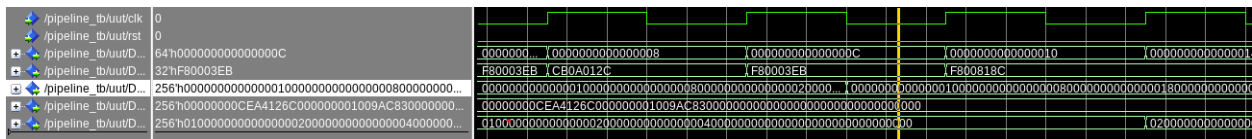
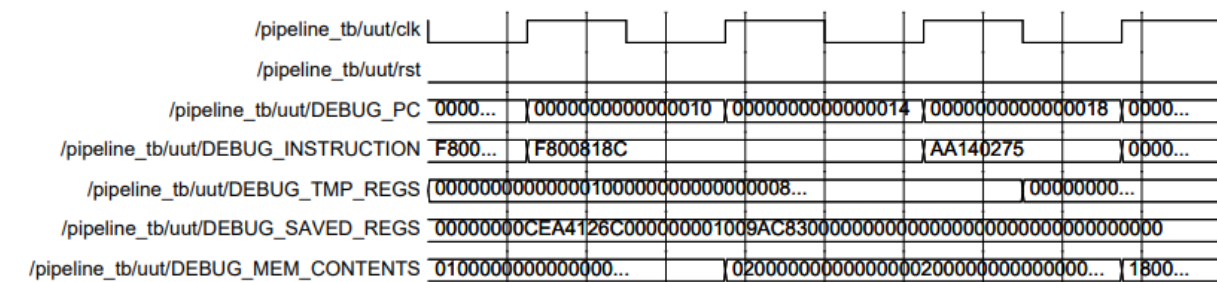
The main object of this lab is to build a pipelined CPU consists of the components that I mentioned in the “README” file. In this report I’ll give out the waveform of the signals and explain some of the most important signals to show how the pipelined CPU works. To make sure that the signals are clearly shown, I will break the process into four figures, each with 2000 ns.



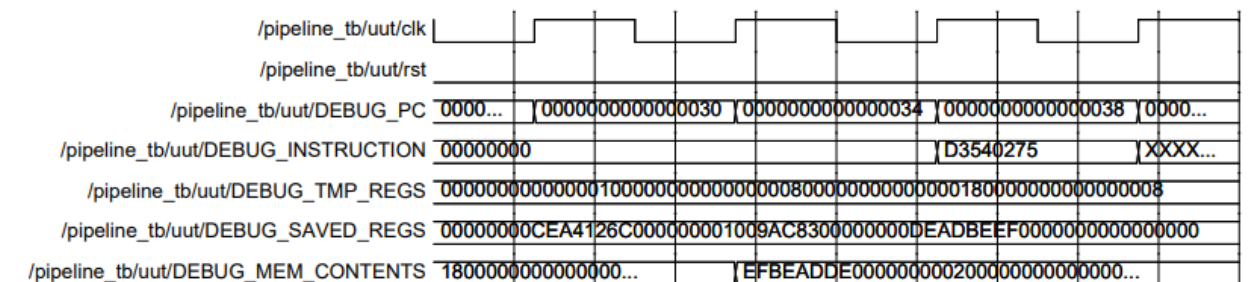
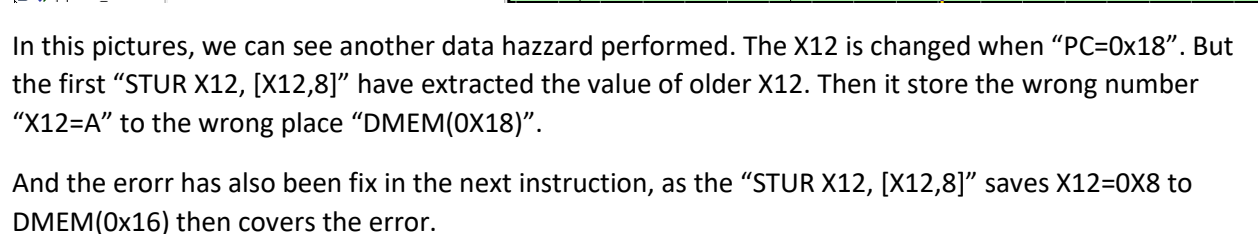
Right from the beginning, we can see that the signal `DEBUG_PC` is correctly operating. And I will use that signal as a base to show how the instructions work.

The `DEBUG_TMP_REGS` signal shows what X9, X10, X11 and X12 contain and `DEBUG_SAVED_REGS` shows what X19, X20, X21 and X22 contains. The `DEBUG_MEM_CONTENTS` gives the contents saved in data memory from the location “0x0” to “0x28”.

When PC output is just zero, the instruction memory immediately gives out the first instruction. But the registers and the data memory are not changed. This is because the instruction is cut into different stages. Data memory will change only three stages after the “store” instruction is performed and registers takes even longer time as four stages.



In the second picture, we can see that the X11 is changed when PC goes to "0C". However, the instruction "STUR X11, XZR, 0" is already performed and the original value of "X11=0x02" has been extracted. Thus, there is a data hazard in the instruction" STUR X11, XZR, 0" and the value stored in data



The final picture shows how the instructions “ORR X21, X19, X20” and “STUR X21, [XZR, 0]” are successfully performed without data hazard. This is because the NOP is inserted between those two instructions.